Programmation Frontend

Lab 2bis — L'état React en profondeur

Re-rendu, immutabilité et gestion des données

ESP/UCAD — 2025–2026

Dr. El Hadji Bassirou TOURÉ
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Informations pratiques

Durée estimée	2h (une séance)
Prérequis	Avoir terminé le Lab 2 (composants, props, CSS Modules).
Fichiers de départ	Le projet `annuaire-react/` du Lab 2
Livrables	Annuaire avec tri A-Z/Z-A, bouton favoris sur chaque carte, compteur de favoris
Validation	Le tri fonctionne, les favoris se togglent, le compteur se met à jour

Objectifs du lab

À l'issue de ce lab, chaque étudiant sera capable de :

1. Expliquer **pourquoi** une variable `let` ne suffit pas pour gérer des données qui changent dans React
2. Décrire ce qu'est un **re-rendu** et quand il se produit
3. Distinguer **état** et **props**
4. Gérer des **états multiples** indépendants dans un même composant
5. Appliquer la règle d'**immutabilité** pour modifier un tableau dans l'état

Table des matières

1 Point de départ — Le code du Lab 2 **3**

2 Partie 1 — Pourquoi `let` ne marche pas **3**
2.1 Le test : un compteur avec `let` . . . 3
2.2 Pourquoi ? . . . 4
2.3 Le même compteur avec `useState` . . . 4

3 Partie 2 — Le re-rendu **5**
3.1 Voir le re-rendu . . . 5

4 Partie 3 — État vs Props **6**

5 Partie 4 — États multiples : ajouter un tri **6**
5.1 Un composant peut avoir plusieurs états 6
5.2 Exercice 1 — Bouton de tri A-Z / Z-A 7

6 Partie 5 — Immutabilité : ne jamais modifier l'état directement **9**
6.1 Le piège : `push` ne déclenche rien 9
6.2 Les bons patterns 9

7 Partie 6 — Exercice : les favoris **10**
7.1 Le principe 10
7.2 Modifier `App.jsx` 10
7.3 Modifier `UserCard.jsx` 11
7.4 Tester 11

8 Aide-mémoire **12**

9 Résumé et conclusion **13**

1 Point de départ — Le code du Lab 2

Au Lab 2, vous avez découpé l'annuaire en composants (`App`, `UserCard`, `SearchBar`) avec des CSS Modules. Vous avez utilisé `useState` pour stocker les utilisateurs, le texte de recherche et l'état de chargement.

Mais on n'a jamais expliqué **pourquoi** il faut `useState`. Pourquoi ne pas utiliser une simple variable `let` ? Pourquoi appeler `setUsers(data)` au lieu de `users = data` ?

Ce lab répond à cette question.

Étape 0 — Vérification

Ouvrez le projet `annuaire-react/` dans VS Code. Lancez le serveur :

```
cd annuaire-react
npm run dev
```

Ouvrez `http://localhost:5173`. Cliquez sur le bouton. Les 10 utilisateurs apparaissent. Le filtre fonctionne.

2 Partie 1 — Pourquoi `let` ne marche pas

Objectif — 15 minutes

Comprendre la différence fondamentale entre une variable JavaScript et un état React.

2.1 Le test : un compteur avec `let`

Créez un fichier temporaire `src/Compteur.jsx` :

```
const Compteur = () => {
  let count = 0;

  const incrementer = () => {
    count = count + 1;
    console.log("count vaut :", count);
  };

  return (
    <div>
      <p>Compteur : {count}</p>
      <button onClick={incrementer}>+1</button>
    </div>
  );
};

export default Compteur;
```

Pour tester, modifiez temporairement `src/App.jsx` en ajoutant l'import et en affichant le composant :

```
import Compteur from "./Compteur";

// Dans le return, ajoutez en haut :
<Compteur />
```

Cliquez sur le bouton **+1** plusieurs fois. Regardez :

- La **console** affiche `count vaut : 1, 2, 3...` La variable change bien.
- L'**écran** affiche toujours `Compteur : 0`. L'affichage ne bouge pas.

2.2 Pourquoi ?

React ne surveille **pas** les variables JavaScript classiques. Quand vous modifiez `count` avec `count = count + 1`, JavaScript met à jour la variable en mémoire, mais **React ne le sait pas**. Il ne re-rend pas le composant. L'écran reste figé.

2.3 Le même compteur avec `useState`

Modifiez `src/Compteur.jsx` :

```
import { useState } from "react";

const Compteur = () => {
  const [count, setCount] = useState(0);

  const incrementer = () => {
    setCount(count + 1);
    console.log("count vaut :", count);
  };

  return (
    <div>
      <p>Compteur : {count}</p>
      <button onClick={incrementer}>+1</button>
    </div>
  );
};

export default Compteur;
```

Cliquez sur **+1**. L'écran se met à jour à chaque clic. `useState` permet à React de **détecter** le changement et de **re-rendre** le composant.

Variable `let` vs état `useState`

	`let count = 0`	`useState(0)`
Modification	`count = count + 1`	`setCount(count + 1)`
React le sait ?	Non	Oui
Re-rendu ?	Non — l'écran reste figé	Oui — l'écran se met à jour
Survit au re-rendu ?	Non — remis à 0	Oui — React conserve la valeur

La console affiche l'ancienne valeur

Dans la version `useState`, la console affiche un coup de retard : quand `count` vaut 0 et qu'on appelle `setCount(1)`, le `console.log` affiche encore 0. C'est normal : `setCount` **planifie** un re-rendu avec la nouvelle valeur, mais la variable `count` dans le rendu actuel ne change pas. La nouvelle valeur sera visible au prochain rendu.

3 Partie 2 — Le re-rendu

Objectif — 10 minutes

Comprendre ce qui se passe quand un état change.

3.1 Voir le re-rendu

Ajoutez un `console.log` **dans le corps du composant** (en dehors de toute fonction) :

```
const Compteur = () => {
  const [count, setCount] = useState(0);

  console.log("Rendu du composant ! count =", count);

  const incrementer = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <p>Compteur : {count}</p>
      <button onClick={incrementer}>+1</button>
    </div>
  );
};
```

Ouvrez la console du navigateur (`F12`). Cliquez sur **+1** plusieurs fois. Vous verrez :

```
Rendu du composant ! count = 0
Rendu du composant ! count = 1
Rendu du composant ! count = 2
Rendu du composant ! count = 3
```

À chaque clic, React **ré-exécute toute la fonction** `Compteur` avec la nouvelle valeur de `count`. C'est le **re-rendu**.

Analogie : la photo de classe

Imaginez que chaque re-rendu est une **nouvelle photo de classe**. À chaque changement (un élève arrive, un autre part), on ne retouche pas l'ancienne photo — on en prend une nouvelle. React fonctionne pareil : il ne modifie pas l'ancien affichage, il en **génère un nouveau** à partir de l'état actuel.

Le cycle du re-rendu

1. L'utilisateur clique sur un bouton (événement).
2. Le gestionnaire appelle `setCount(count + 1)` (mise à jour de l'état).
3. React ré-exécute la fonction du composant avec la nouvelle valeur (re-rendu).
4. React compare l'ancien résultat et le nouveau, puis met à jour **uniquement** ce qui a changé dans le DOM.

4 Partie 3 — État vs Props

Objectif — 10 minutes

Distinguer clairement l'état et les props.

Au Lab 2, vous avez appris les **props** : des données envoyées par le parent. Maintenant, vous comprenez l'**état** : des données internes au composant. Les deux sont les mécanismes fondamentaux de React pour gérer les données.

État vs Props

	État (`useState`)	**Props**
Qui le crée ?	Le composant lui-même	Le composant parent
Qui le modifie ?	Le composant (via `set...`)	Personne — lecture seule
Objectif	Données internes qui changent	Données reçues de l'extérieur
Déclenche un re-rendu ?	Oui (quand on appelle `set...`)	Oui (quand le parent re-rend)

Analogie : le sac et la fiche

L'état, c'est ce que vous avez **dans votre sac**. Vous décidez quoi y mettre, quoi en retirer. Personne d'autre ne touche à votre sac.
Les props, c'est une **fiche de commande** qu'on vous donne. Vous la lisez, vous l'utilisez, mais vous ne la modifiez pas. Si le contenu de la fiche change, c'est celui qui l'a écrite qui la met à jour.

Dans l'annuaire :

- `users` est un **état** de `App` (App le crée, App le modifie).
- `user` est une **prop** de `UserCard` (App l'envoie, UserCard le lit).
- `recherche` est un **état** de `App`, et une **prop** de `SearchBar`.

Le dernier point est important : une même donnée peut être un état dans un composant et une prop dans un autre. L'état vit là où il est créé. Il descend vers les enfants sous forme de props.

Vous pouvez maintenant supprimer `src/Compteur.jsx` et retirer son import de `App.jsx`. Il a servi de terrain d'expérimentation.

5 Partie 4 — États multiples : ajouter un tri

Objectif — 15 minutes

Ajouter un bouton de tri à l'annuaire pour illustrer les états multiples.

5.1 Un composant peut avoir plusieurs états

Chaque appel à `useState` crée un état **indépendant**. Dans `App.jsx`, on a déjà trois états : `users`, `chargement` et `recherche`. On va en ajouter un quatrième : `triAscendant`.

5.2 Exercice 1 — Bouton de tri A-Z / Z-A

Modifiez `src/App.jsx`. Ajoutez l'état et le bouton :

```
import { useState } from "react";
import UserCard from "./components/UserCard";
import SearchBar from "./components/SearchBar";
import styles from "./App.module.css";

const App = () => {
  const [users, setUsers] = useState([]);
  const [chargement, setChargement] = useState(false);
  const [recherche, setRecherche] = useState("");
  const [triAscendant, setTriAscendant] = useState(true);

  const chargerUtilisateurs = async () => {
    setChargement(true);
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );
      const data = await response.json();
      setUsers(data);
    } catch (error) {
      console.error("Erreur :", error);
    }
    setChargement(false);
  };

  const usersFiltres = users
    .filter(user =>
      user.name.toLowerCase().includes(
        recherche.toLowerCase()
      )
    )
    .sort((a, b) => {
      if (triAscendant) {
        return a.name.localeCompare(b.name);
      }
      return b.name.localeCompare(a.name);
    });

  return (
    <div className={styles.container}>
      <h1 className={styles.titre}>
        Annuaire des utilisateurs
      </h1>
      <button className={styles.bouton}
              onClick={chargerUtilisateurs}
              disabled={chargement}>
        {chargement ? "Chargement..." : "Charger"}
      </button>

      <SearchBar
        recherche={recherche}
        onRecherche={setRecherche}
      />

      <div className={styles.barre}>
```

```
        <p className={styles.compteur}>
          {usersFiltres.length} utilisateur(s)
        </p>
        <button className={styles.boutonTri}
                onClick={() => setTriAscendant(!triAscendant)}>
          Tri : {triAscendant ? "A → Z" : "Z → A"}
        </button>
      </div>

      {usersFiltres.map(user => (
        <UserCard key={user.id} user={user} />
      ))}
    </div>
  );
};

export default App;
```

Ajoutez les classes dans `src/App.module.css` :

```
.barre {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.boutonTri {
  padding: 0.3rem 0.8rem;
  font-size: 0.85rem;
  cursor: pointer;
  border: 1px solid #ccc;
  border-radius: 6px;
  background: #f9f9f9;
}

.boutonTri:hover {
  background: #eee;
}
```

Sauvegardez. Chargez les utilisateurs, puis cliquez sur le bouton de tri. La liste se réordonne instantanément. Chaque clic bascule entre A→Z et Z→A.

Plusieurs états — chacun indépendant

L'application a maintenant 4 états dans `App` : `users`, `chargement`, `recherche`, `triAscendant`. Quand on clique sur le bouton de tri, seul `triAscendant` change. React re-rend `App`, recalcule `usersFiltres` (avec le nouveau tri), et met à jour l'affichage. Les autres états ne bougent pas.

6 Partie 5 — Immutabilité : ne jamais modifier l'état directement

Objectif — 15 minutes

Comprendre la règle d'immutabilité et savoir ajouter/supprimer des éléments dans un tableau d'état.

6.1 Le piège : `push` ne déclenche rien

Imaginons que vous vouliez ajouter un utilisateur à la liste. Votre premier réflexe :

```
// NE FONCTIONNE PAS
users.push(nouvelUtilisateur);
setUsers(users);
```

L'écran ne se met **pas** à jour. Pourquoi ? Parce que `push` modifie le tableau existant. Quand on appelle `setUsers(users)`, React compare l'ancien tableau et le nouveau : c'est **le même objet en mémoire**. React conclut que rien n'a changé et ne re-rend pas.

La règle d'or

Ne modifiez jamais un état directement. Pas de `push`, pas de `splice`, pas de `users[0] = ...`. Créez toujours une **copie** avec les modifications souhaitées.

6.2 Les bons patterns

Action	Mauvais ✗	Bon ✓
Ajouter	`arr.push(x);` `setArr(arr);`	`setArr([...arr, x]);`
Supprimer	`arr.splice(i, 1);` `setArr(arr);`	`setArr(arr.filter(...))`
Modifier	`arr[i].name = "X";` `setArr(arr);`	`setArr(arr.map(...))`

L'opérateur `...` (*spread*) crée une **copie superficielle** du tableau. `[...arr, x]` signifie : « un nouveau tableau contenant tous les éléments de `arr` plus `x` à la fin ».

Analogie : la liste de courses

Imaginez que votre liste de courses est écrite au stylo indélébile. Vous ne pouvez pas barrer un article ni en ajouter un. À la place, vous **recopiez** toute la liste sur une nouvelle feuille en ajoutant ou en retirant ce que vous voulez. C'est plus de travail, mais vous avez toujours une trace de l'ancienne liste (l'ancien état).

7 Partie 6 — Exercice : les favoris

Objectif — 15 minutes

Ajouter un système de favoris à l'annuaire. Cet exercice met en pratique les états multiples et l'immutabilité.

7.1 Le principe

On veut :

- Un bouton ★ (étoile) sur chaque carte.
- Cliquer sur l'étoile ajoute l'utilisateur aux favoris. Recliquer le retire.
- Un compteur en haut indique le nombre de favoris.

L'état `favoris` est un **tableau d'identifiants** stocké dans `App`.

7.2 Modifier `App.jsx`

Ajoutez l'état et la fonction de toggle dans `src/App.jsx`. Voici les lignes à ajouter (le reste ne change pas) :

```
// Après les autres useState
const [favoris, setFavoris] = useState([]);

// Nouvelle fonction
const toggleFavori = (userId) => {
  if (favoris.includes(userId)) {
    // Retirer : garder tous les ids SAUF celui-ci
    setFavoris(favoris.filter(id => id !== userId));
  } else {
    // Ajouter : copier le tableau + ajouter l'id
    setFavoris([...favoris, userId]);
  }
};
```

Dans le `return`, ajoutez le compteur après la barre de tri et passez les nouvelles props à `UserCard` :

```
{favoris.length > 0 && (
  <p className={styles.favoris}>
    {favoris.length} favori(s)
  </p>
)}

{usersFiltres.map(user => (
  <UserCard
    key={user.id}
    user={user}
    estFavori={favoris.includes(user.id)}
    onToggleFavori={toggleFavori}
  />
))}
```

Ajoutez la classe dans `src/App.module.css` :

```
.favoris {
  color: #e67e22;
  font-weight: bold;
```

```
  font-size: 0.9rem;
  margin: 0.3rem 0;
}
```

7.3 Modifier `UserCard.jsx`

Modifiez `src/components/UserCard.jsx` pour afficher le bouton étoile :

```
import styles from "./UserCard.module.css";

const UserCard = ({ user, estFavori, onToggleFavori }) => {
  return (
    <div className={styles.card}>
      <div className={styles.header}>
        <h3 className={styles.nom}>{user.name}</h3>
        <button className={styles.etoile}
                onClick={() => onToggleFavori(user.id)}>
          {estFavori ? "\u2605" : "\u2606"}
        </button>
      </div>
      <p className={styles.info}>Email : {user.email}</p>
      <p className={styles.info}>
        Entreprise : {user.company.name}
      </p>
    </div>
  );
};

export default UserCard;
```

Le caractère `\u2605` est une étoile pleine (★), `\u2606` est une étoile vide.

Ajoutez les classes dans `src/components/UserCard.module.css` :

```
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.etoile {
  background: none;
  border: none;
  font-size: 1.5rem;
  cursor: pointer;
  color: #e67e22;
  padding: 0;
  line-height: 1;
}
```

7.4 Tester

Sauvegardez tout. Chargez les utilisateurs. Cliquez sur les étoiles : elles se remplissent et se vident. Le compteur de favoris en haut se met à jour.

Vérifiez aussi que le **tri** et le **filtre** fonctionnent toujours avec les favoris.

Ce que l'exercice met en pratique

Concept	Où dans le code
État tableau	`const [favoris, setFavoris] = useState([])`
Immutabilité (ajout)	`setFavoris([...favoris, userId])`
Immutabilité (retrait)	`setFavoris(favoris.filter(id => id !== userId))`
Prop fonction	`onToggleFavori={toggleFavori}`
Rendu conditionnel	`estFavori ? "★" : "étoile vide"`

8 Aide-mémoire

Concept	Syntaxe / Exemple
Créer un état	`const [val, setVal] = useState(initial);`
Modifier un état	`setVal(nouvelleValeur);`
État booléen	`setActif(!actif);`
Ajouter à un tableau	`setArr([...arr, nouvelElement]);`
Retirer d'un tableau	`setArr(arr.filter(x => x !== cible));`
Modifier dans un tableau	`setArr(arr.map(x => x.id === id ? {...x, nom: "Y"} : x));`
Re-rendu	Se produit quand `set...` est appelé
Immutabilité	Ne jamais modifier l'état directement (`push`, `splice`)
État vs props	État = interne (modifiable). Props = externe (lecture seule).

9 Résumé et conclusion

Dans ce lab, vous avez :

1. Compris **pourquoi `let` ne suffit pas** : React ne surveille pas les variables classiques.
2. Observé le **re-rendu** : quand un état change, React ré-exécute le composant.
3. Distingué **état et props** : l'état est interne et modifiable, les props sont externes et en lecture seule.
4. Géré des **états multiples** indépendants (tri A-Z/Z-A).
5. Appliqué la règle d'**immutabilité** avec le spread operator (`...`) et `.filter()`.
6. Implémenté un système de **favoris** complet.

Voici où vous en êtes dans la progression du cours :

Étape	Statut	Ce que vous savez faire
DOM et événements	✓	Sélecteurs, événements, modification du DOM
Fonctions JS	✓	Déclaration, expression, fléchée
Asynchrone	✓	`fetch`, `async/await`
React — les bases	✓	Composant, JSX, Vite, `.map()`
Composants et props	✓	Découpage, props, CSS Modules
État React	✓	`useState`, re-rendu, immutabilité, état vs props
useEffect	→ Lab 3	Charger au montage, loading, détail
Formulaires et routing	→ Lab 4	Inputs contrôlés, React Router

Et ensuite ?

Vous maîtrisez maintenant les deux mécanismes fondamentaux de React : l'**état** (données internes) et les **props** (données reçues). Dans le **Lab 3**, vous apprendrez `useEffect` pour exécuter du code « à côté » du rendu : charger les données automatiquement au démarrage, gérer le loading et les erreurs, et afficher une vue de détail.

Exercice bonus — Pour aller plus loin

Bonus : afficher uniquement les favoris

Si vous avez terminé en avance, ajoutez un bouton « Voir mes favoris » qui filtre la liste pour n'afficher que les utilisateurs marqués comme favoris. Indices :

1. Ajoutez un état `const [voirFavoris, setVoirFavoris] = useState(false);`
2. Ajoutez un bouton qui bascule cet état.
3. Dans le calcul de `usersFiltres`, ajoutez une condition :
   `.filter(user => !voirFavoris || favoris.includes(user.id))`
4. Le bouton affiche « Voir mes favoris » ou « Voir tous » selon l'état.

Lab 2bis — L'état React — Terminé ✓

Vous savez pourquoi `useState` existe, ce qu'est un re-rendu,
et comment modifier un état sans enfreindre l'immutabilité.

Prochaine étape : `useEffect` et chargement automatique.

Modifier l'état = prendre une nouvelle photo.